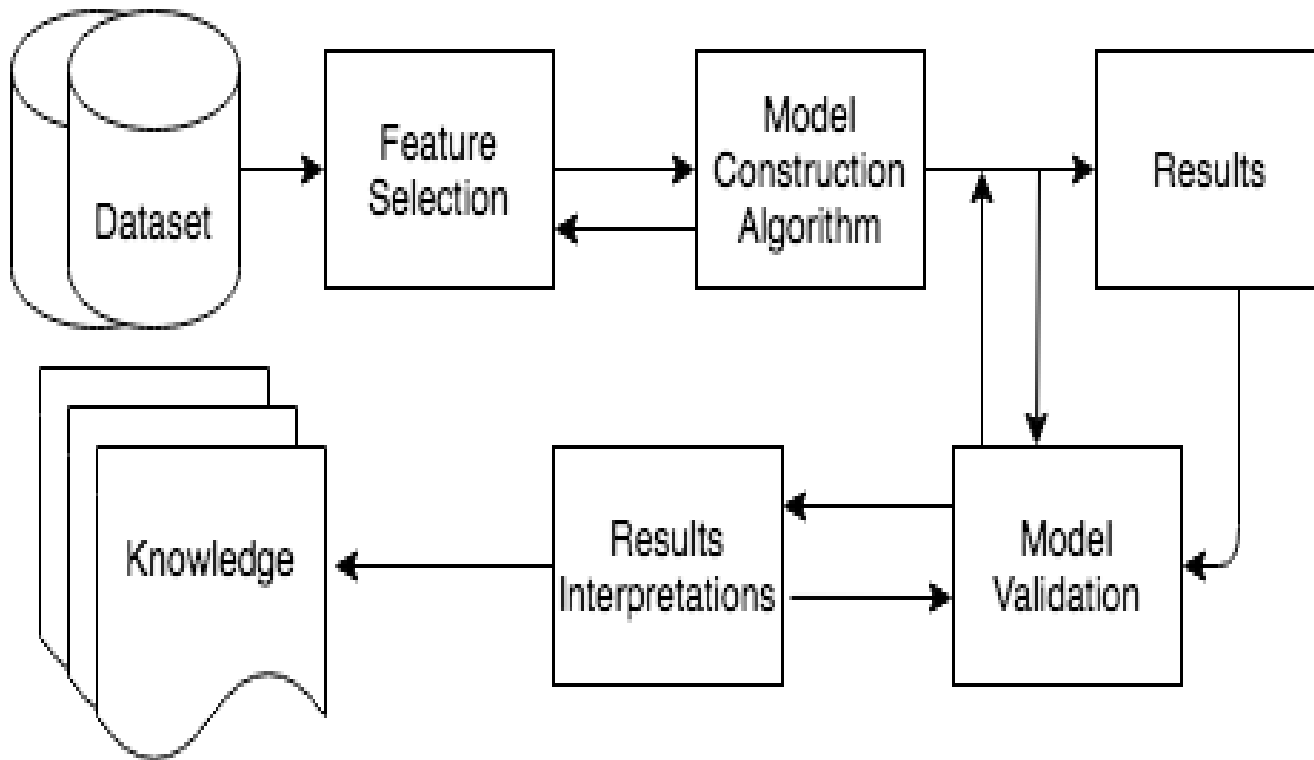


Classification Model

- Classification: Basic Concepts
- Rule-Based Classification
- Decision Tree Induction 
- Bayes Classification Methods
- kNN Classifier
- Performance Evaluation
- Ensemble Classification
- Summary

Basic Steps of Data Analysis



Decision Tree Model

Decision Tree

- In principle, there are **exponentially many decision trees** that can be constructed from a given set of attributes.
- Finding the optimal tree (in terms of performance) is computationally **infeasible** because of the exponential size of the search space.
- Efficient algorithms have been developed to induce a reasonably **accurate** although **suboptimal** decision tree in a reasonable amount of time.
- These algorithms usually employ a **greedy strategy** that grows a decision tree by making a series of locally optimum decision about which attribute to use for partitioning the data.
- One such algorithm is Hunt's algorithm, which is the basis of many decision tree induction algorithms, including ID3, C4.5, and CART.

Hunt's Algorithm

- In this algorithm, a decision tree is grown in a recursive fashion by partitioning the training records into successively purer subsets.
- Let D_t be the set of training records that are associated with node t and $y = \{y_1, y_2, \dots, y_c\}$ be the class labels. The following is a recursive definition of Hunt's algorithm:
 - Step 1: If all the records in D_t belong to the same class y_t , then t is a leaf node labelled as y_t .
 - Step 2: If D_t contains records that belong to more than one class, an **attribute test condition** is selected to partition the records into smaller subsets. A child node is created for each outcome of the test condition and the records in D_t are distributed to the children based on the outcomes.
 - The algorithm is then recursively applied to each child node.

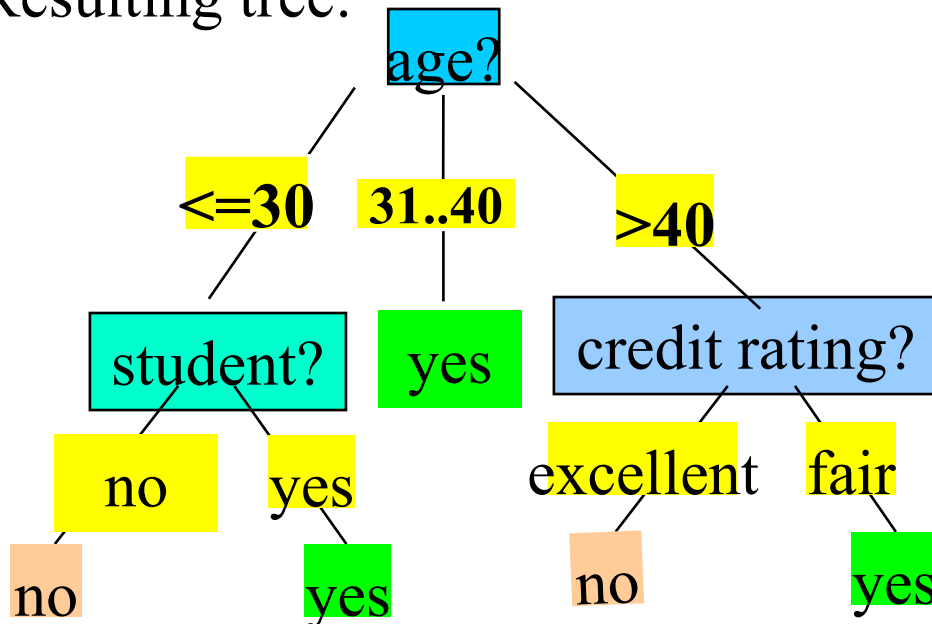
Decision Tree Induction: ID3

- ❑ Training data set: Buys_computer
- ❑ Decision tree model is constructed using the **Quinlan ID3** algorithm.
- ❑ ID3 refers to Iterative Dichotomizer 3, and is developed by Ross Quinlan
- ❑ It iteratively dichotomizes (divides) the data using a top down greedy approach.
- ❑ The algorithm optimizes locally at each iteration.

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Decision Tree Induction: ID3

- Root is identified first, then the parents of next label, and so on of the decision tree are identified using entropy theory based optimization function
- Resulting tree:



age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Algorithm of ID3

- Basic algorithm (a greedy algorithm)
 - Tree is constructed in a **top-down recursive divide-and-conquer manner**
 - At start, all the training examples are at the root
 - Attributes are categorical (if continuous-valued, they are discretized in advance)
 - Examples are partitioned recursively based on selected attributes
 - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., **information gain**)
- Conditions for stopping partitioning
 - All samples for a given node belong to the same class
 - There are no remaining attributes for further partitioning – **majority voting** is employed for classifying the leaf
 - There are no samples left

Brief Review of Entropy

(Already discussed in entropy based discretization technique)

■ Entropy (Information Theory)

- A measure of uncertainty associated with a random variable

- Calculation: For a discrete random variable Y taking m distinct values $\{y_1, \dots, y_m\}$,

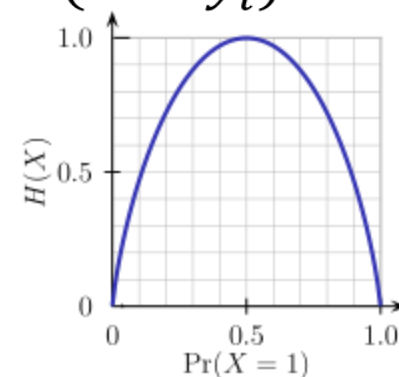
- $H(Y) = -\sum_{i=1}^m p_i \log(p_i)$, where $p_i = P(Y = y_i)$

- Interpretation:

- Higher entropy => higher uncertainty
 - Lower entropy => lower uncertainty

■ Conditional Entropy

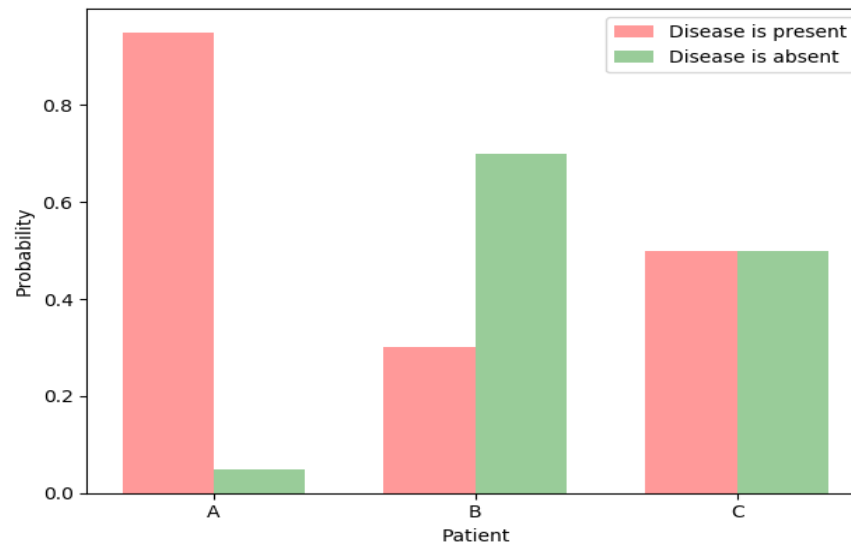
- $H(Y|X) = \sum_x p(x)H(Y|X = x)$



$Y/(X=x_1)$	$Y/(X=x_2)$		$Y/(X=x)$	$Y/(X=x_n)$

Entropy and uncertainty

- Suppose three patients have completed a medical test which, yields one of two possible results: the disease is either present or absent.
- Let Patient A has 95% chance that he has the disease. For Patient B and C it is 30% and 50%, respectively.
- So, if A, B, and C are in waiting room of a doctor's office, then the uncertainty of the waiting room is:



Entropy and uncertainty

- All other things being equal, which of the three patients is confronted with the greatest degree of uncertainty?
- I think the answer is clear: patient C.
- Compare this with patient A. Patient A is experiencing little uncertainty with regard to his medical prospects.
- Intuitively speaking, uncertainty of patient B falls in between that of A and C.
- **Measuring uncertainty:** Entropy is a measure of uncertainty.
- Entropy allows us to make precise statements and perform computations with regard to one of life's most pressing issues: not knowing how things will turn out.
- By the term entropy, I will refer to **Shannon entropy**, which is used most frequently in natural language processing and machine learning.

Entropy and uncertainty

- The **Shannon entropy** formula for an event X with n possible outcomes and probabilities $p_1, \dots, p_n$:

$$H(X) = H(p_1, \dots, p_n) = - \sum_{i=1}^n p_i \log_2 p_i$$

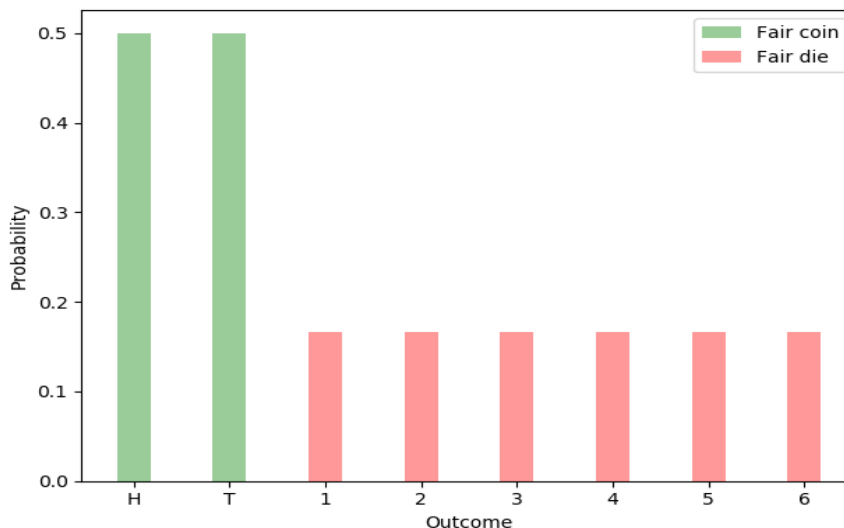
- **Basic properties of uncertainty:**

Prop. 1: Uniform distributions have maximum uncertainty

If your goal is to minimize uncertainty, stay away from **uniform probability distributions**.

Prop -1 of uncertainty

- A probability distribution is a function that assigns a probability to every possible outcome such that the probabilities add up to 1.
- A distribution is uniform when all of the outcomes have the same probability.
- For example, fair coins (50% heads, 50% tails) and fair dice (1/6 probability for each of the six faces) follow uniform distributions.



Prop -1 of uncertainty

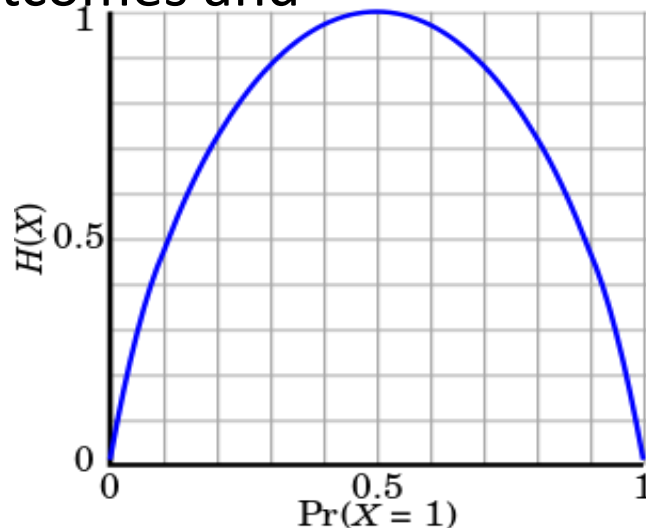
- A good measure of uncertainty achieves its highest values for uniform distributions.

- Entropy satisfies the criterion. Given n possible outcomes, maximum entropy is maximized by equiprobable outcomes:

$$p_1 = \dots = p_n = \frac{1}{n}$$

- Here is the plot of the Entropy function as applied to Bernoulli trials (events with two possible outcomes and probabilities p and $1-p$):

- In the case of Bernoulli trials, entropy reaches its maximum value for $p=0.5$



Prop -2 of uncertainty

- **Prop-2: Uncertainty is additive for independent events**
- Let A and B be independent events. In other words, knowing the outcome of event A does not tell us anything about the outcome of event B .
- The uncertainty associated with both events should be the sum of the individual uncertainties:

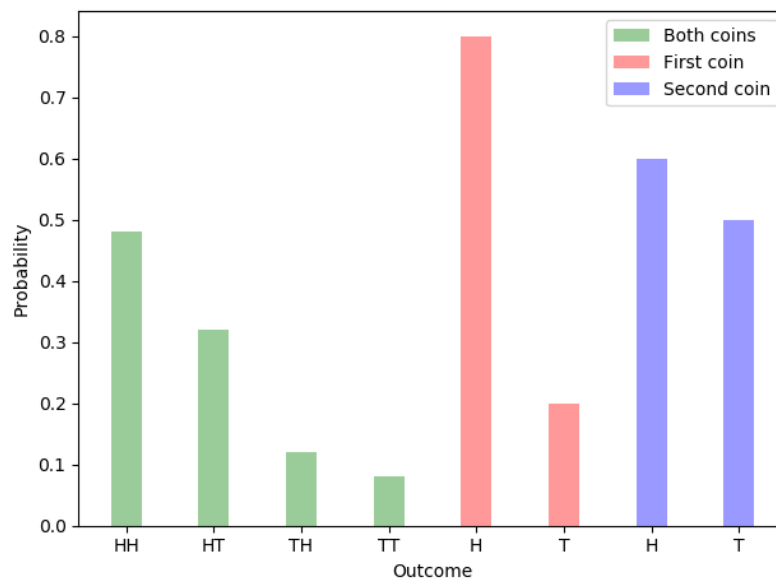
$$H(X, Y) = H(X) + H(Y)$$

- Let's use the example of flipping two coins to make this more concrete. We can either flip both coins simultaneously or first flip one coin and then flip the other one.
- In other words, we can either report the outcome of the two coin flips at once or separately. The uncertainty is the same in either case.

Prop -2 of uncertainty

- Ex: Let the first coin lands heads (H) up with an 80% probability and tails (T) up with a probability of 20%.
- The probabilities for the other coin are 60% and 40%.
- If we flip both coins simultaneously, there are four possible outcomes: HH , HT , TH and TT . The corresponding probabilities are given by $[0.48, 0.32, 0.12, 0.08]$.

- The joint entropy (green) for the two independent events is equal to the sum of the individual events (red and blue).



Prop -2 of uncertainty

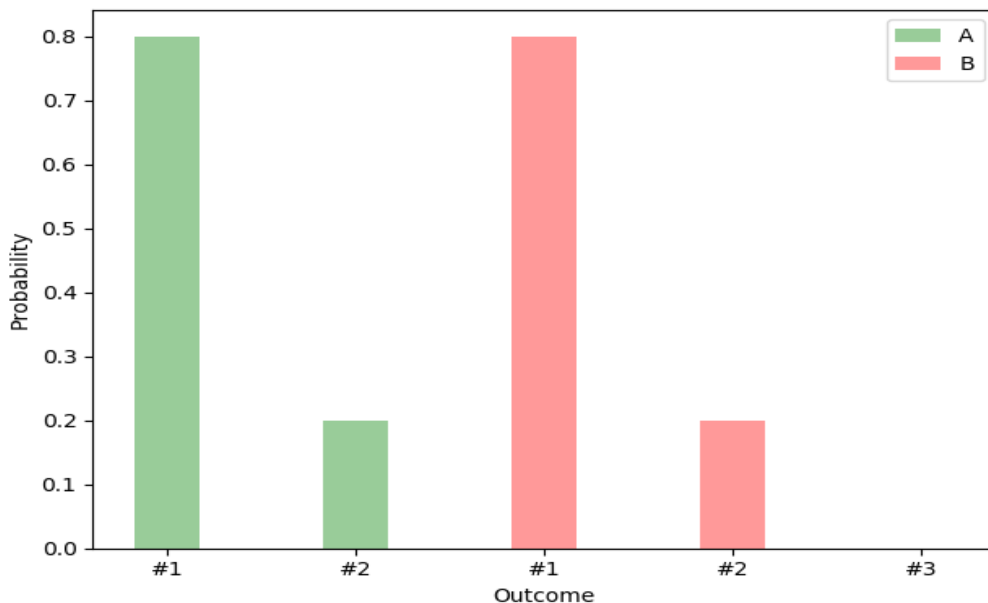
- Plugging the numbers into the entropy formula, we see that: Just as promised,

$$H(0.48, 0.32, 0.12, 0.08) = H(0.8, 0.2) + H(0.6, 0.4)$$

Prop -3 of uncertainty

Prop-3: Adding an outcome with zero probability has no effect

- Suppose (a) you win whenever outcome #1 occurs and (b) you can choose between two probability distributions, *A* and *B*.
- Distribution *A* has two outcomes: say, 80% and 20%. Distribution *B* has three outcomes with probabilities 80%, 20% and 0%.



Prop -3 of uncertainty

- Given the options A and B , which one would you choose?
- Ans: The inclusion of the third outcome neither increases nor decreases the uncertainty associated with the game. A or B , who cares. It doesn't matter.
- The entropy formula agrees with this assessment:
$$H(p_1, \dots, p_n) = H(p_1, \dots, p_n, 0)$$
- In words, adding an outcome with zero probability has no effect on the measurement of uncertainty.

Prop -4 of uncertainty

Prop-4: The measure of uncertainty is continuous in all its arguments

- The last of the basic properties is continuity.
- Famously, the intuitive explanation of a continuous function is that arbitrarily small changes in the output (uncertainty, in our case) should be achievable through sufficiently small changes in the input (probabilities).
- Logarithm functions are continuous at every point for which they are defined. So are sums and products of a finite number of functions that are continuous on a subset.
- It follows that the entropy function is continuous in its probability arguments.

Entropy

- **The Uniqueness Theorem:** [Khinchin \(1957\)](#) showed that the only family of functions satisfying the four basic properties described above is of the following form:

$$H(p_1, \dots, p_n) = -\lambda \sum_{i=1}^n p_i \log p_i,$$

- where λ is a positive constant. Khinchin referred to this as the **Uniqueness Theorem**.
- Setting $\lambda = 1$ and using the binary logarithm gives us the Shannon entropy.
- To reiterate, entropy is used because it has desirable properties and is the natural choice among the family functions that satisfy all items on the basic properties.

Entropy

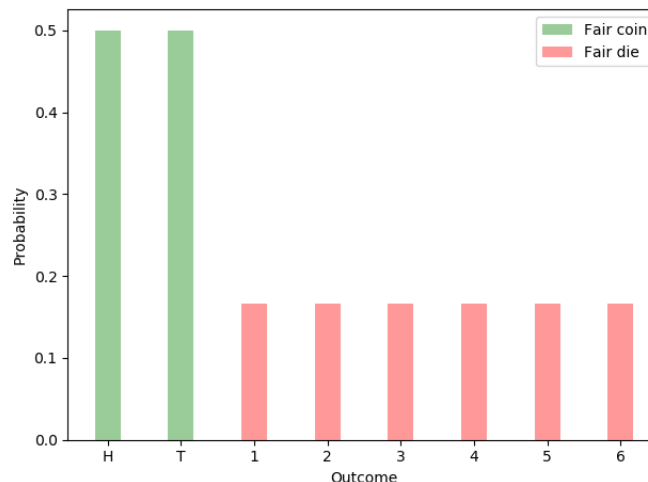
Other properties of Entropy:

- Entropy has many other properties used in Khinchin's Uniqueness Theorem. Some of them:

Prop-5: Uniform distributions with more outcomes have more uncertainty

- Suppose you have the choice between a fair coin and a fair die:

Fair coin or fair die ?



Entropy

- And let's say you win if the coin lands heads up or the die lands on face 1.
- Which of the two options would you choose?
 - (i) if you are a profit maximizer and
 - (ii) if you prefer with more variety and uncertainty.
- As the number of equiprobable outcomes increases, so should our measure of uncertainty.
- And this is exactly what Entropy does: $H(1/6, 1/6, 1/6, 1/6, 1/6, 1/6) > H(0.5, 0.5)$.
- And, in general, if we let $L(k)$ be the entropy of a uniform distribution with k possible outcomes,

$$L(m) > L(n) \quad \text{for } m > n$$

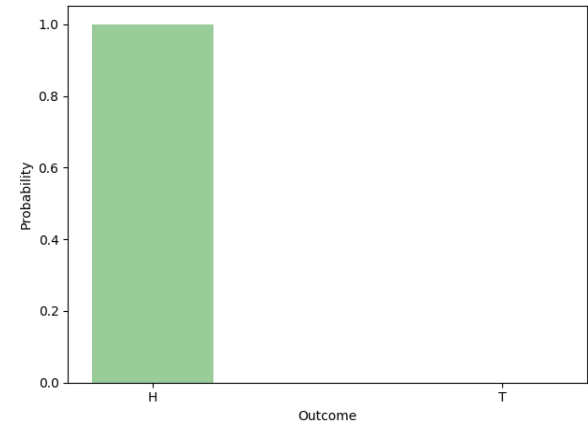
Entropy

Prop-6: Events have non-negative uncertainty

- Do you know what negative uncertainty is? Neither do I.
- A user-friendly measure of uncertainty should always return a non-negative quantity, no matter what the input is.
- This is yet another criterion that is satisfied by entropy. Let's take another look at the formula: $H(X) = - \sum_{i=1}^n p_i \log_2 p_i$
- Probabilities are, by definition, in the range between 0 and 1 and, therefore, non-negative.
- The logarithm of a probability is non-positive. Multiplying the logarithm of a probability with a probability doesn't change the sign. The sum of non-positive products is non-positive. And finally, the negative of a non-positive value is non-negative.
- Entropy is, thus, non-negative for every possible input.

Entropy

- **Prop-7: Events with a certain outcome have zero uncertainty**
- Suppose you are in possession of a magical coin. No matter how you flip always lands head up.
 - Suppose that outcome i certain to occur. It follows that p_i , the probability of outcome i , is equal to

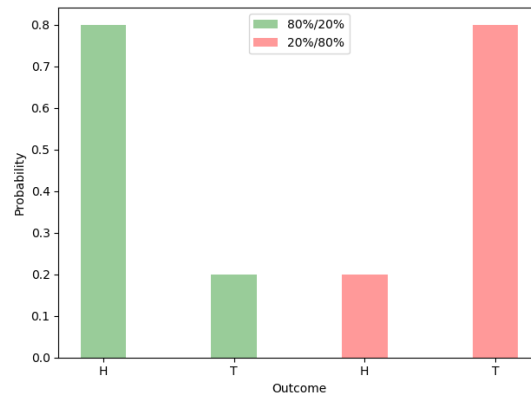


1. $H(X)$, thus, simplifies to:

$$H(0, \dots, 1, \dots, 0) = -\log_2 1 = 0$$

Entropy

- **Prop-8: Flipping the arguments has no effect**
- This is another obviously desirable property. Consider two cases. In the first case, the probability of heads and tails are 80% and 20%. In the second case, the probabilities are reversed: heads 20%, tails 80%.



Entropy

- Both coin flips are equally uncertain and have the same entropy: $H(0.8, 0.2) = H(0.2, 0.8)$.
- In more general terms, for the case of two outcomes, we have:

$$H(p_1, p_2) = H(p_2, p_1)$$

- This fact applies to any number of outcomes. We can position the arguments (i.e., the probabilities of a distribution) in any order we like. The result of the entropy function is always the same.

Attribute Selection Measure: Information Gain (ID3/C4.5)

- Select the attribute with the highest information gain
- Let p_i be the probability that an arbitrary tuple in D belongs to class C_i , estimated by $|C_{i,D}|/|D|$
- Expected information (entropy) needed to classify a tuple in D :

$$Info(D) = -\sum_{i=1}^m p_i \log_2(p_i)$$

- Information needed (after using A to split D into v partitions) to classify a tuple in D :

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

- Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

Attribute Selection: Information Gain

■ Class P: buys_computer = “yes”

■ Class N: buys_computer = “no”

$$Info_{age}(D) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.694$$

$$Info(D) = I(9,5) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0.940$$

age	p _i	n _i	I(p _i , n _i)
<=30	2	3	0.971
31...40	4	0	0
>40	3	2	0.971

$\frac{5}{14} I(2,3)$ means “age <=30” has 5 out of 14 samples, with 2 yes’ es and 3 no’ s. Hence

$$Gain(age) = Info(D) - Info_{age}(D) = 0.246$$

Similarly,

$$Gain(income) = 0.029$$

$$Gain(student) = 0.151$$

$$Gain(credit_rating) = 0.048$$

age	income	student	credit rating	buys computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Computing Information-Gain for Continuous-Valued Attributes

- Let attribute A be a continuous-valued attribute
- Must determine the *best split point* for A
 - Sort the value A in increasing order
 - Typically, the midpoint between each pair of adjacent values is considered as a possible *split point*
 - $(a_i + a_{i+1})/2$ is the midpoint between the values of a_i and a_{i+1}
 - The point with the *minimum expected information requirement* for A is selected as the split-point for A
- Split:
 - D1 is the set of tuples in D satisfying $A \leq \text{split-point}$, and D2 is the set of tuples in D satisfying $A > \text{split-point}$

Gain Ratio

- **Gain Ratio** is an improved version of **Information Gain**,
- In decision trees, it helps mitigate a bias that Information Gain has towards attributes with many values.
- Gain Ratio is often better:

1. Bias in Information Gain

- Information Gain measures how much uncertainty (entropy) is reduced when splitting the data based on a particular attribute.
- However, it tends to favor attributes with a large number of distinct values (e.g., unique IDs, timestamps), even if they are not meaningful for classification.

Gain Ratio

2. Gain Ratio Normalization

- Gain Ratio adjusts Information Gain by dividing it by the **Split Information**, which represents how broadly the data is split.

$$SplitInfo_A(D) = -\sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right)$$

- $GainRatio(A) = Gain(A)/SplitInfo(A)$

Gain Ratio

3. Effect of Gain Ratio

- By penalizing splits that create many small partitions (high Split Information), Gain Ratio avoids favoring attributes with numerous unique values.
- It ensures the selected attribute provides meaningful splits rather than just dividing the data into very small subsets.
- **Example:** Imagine a dataset where: (i) **Attribute A** has 3 categories (low, medium, high) and (ii) **Attribute B** is an ID number (each value is unique).
- If we use **Information Gain**, Attribute B might have the highest score because every split is "pure," but it doesn't generalize well. However, **Gain Ratio** would favor Attribute A because it provides useful, generalizable splits.

Gain Ratio for Attribute Selection (C4.5)

- Gain Ratio improves upon Information Gain by normalizing the effect of many-valued attributes, leading to more balanced and meaningful decision trees.
- It is particularly useful when attributes have significantly different cardinalities.
- C4.5 (a successor of ID3) uses gain ratio to overcome this problem
- Ex. $SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right) = 1.557$
 - $gain_ratio(income) = 0.029/1.557 = 0.019$
- The attribute with the maximum gain ratio is selected as the splitting attribute

CART : Based on Gini Index

- The Gini coefficient measures the [inequality](#) among the values of a [frequency distribution](#), such as levels of [income](#).
- In machine learning, it is utilized as an impurity measure in decision tree algorithms for classification tasks.
- The Gini index is the most commonly used measure of inequality. It was developed by Italian statistician Corrado Gini and is named after him.
- A Gini coefficient of 0 reflects perfect equality, where all income or wealth values are the same.
- A Gini coefficient of 1 (or 100%) reflects maximal inequality among values, a situation where a single individual has all the income while all others have none.

Gini Index

- The **Gini Index**, also known as **Gini Impurity**, is a measure used in decision tree algorithms (like CART – Classification and Regression Trees) to determine the best attribute for splitting data.
- It is calculated especially in terms of class probabilities as:

$$Gini = \sum_{i=1}^n p_i(1 - p_i)$$

- Which is equivalent to $\sum_{i=1}^n p_i - \sum_{i=1}^n p_i^2 = 1 - \sum_{i=1}^n p_i^2$

- Where, n is the number of classes, and p_i is the probability of an instance belonging to class i.

Key Properties of Gini Index

- A **lower Gini Index** indicates a purer node (better classification), while a **higher Gini Index** means more impurity (mixed classes).
- **Range:**
 - The Gini Index ranges from **0 to 1**.
 - $\text{Gini}=0 \rightarrow$ Pure node (all instances belong to one class).
 - $\text{Gini}=1 \rightarrow$ Maximum impurity (instances are evenly distributed among all classes).
- The attribute with the lowest Gini Index is chosen for the split, as it results in purer child nodes.

Gini Index

- For a **binary classification problem**, the Gini Index ranges from **0 to 0.5**, where:
- **0** occurs when all samples belong to one class.
- **0.5** occurs when classes are perfectly split (e.g., 50% in each class).
- Example: Consider a dataset with 2 classes: Yes and No
- Let, there are 10 instances with all 10 of class “Yes” and no instance of class “No”.

Then,

$$Gini = 1 - (p_{Yes}^2 + p_{No}^2)$$
$$= 1 - \left(\left(\frac{10}{10} \right)^2 + \left(\frac{0}{10} \right)^2 \right) = 0$$

Gini Index

- Let, there are 10 instances with 8 of class “Yes” and 2 of class “No”.
Then,

$$\begin{aligned} Gini &= 1 - (p_{Yes}^2 + p_{No}^2) \\ &= 1 - \left(\left(\frac{8}{10} \right)^2 + \left(\frac{2}{10} \right)^2 \right) \\ &= 1 - (17/25) = 8/25 = 0.32 \end{aligned}$$

- Let, there are 10 instances with 6 of class “Yes” and 4 of class “No”.
Then,

$$\begin{aligned} Gini &= 1 - (p_{Yes}^2 + p_{No}^2) \\ &= 1 - \left(\left(\frac{6}{10} \right)^2 + \left(\frac{4}{10} \right)^2 \right) \\ &= 1 - (0.36 + 0.16) = 1 - 0.52 = 0.48 \end{aligned}$$

- Let, there are 10 instances with 5 of class “Yes” and 5 of class “No”.
Then,

$$\begin{aligned} Gini &= 1 - (p_{Yes}^2 + p_{No}^2) \\ &= 1 - \left(\left(\frac{5}{10} \right)^2 + \left(\frac{5}{10} \right)^2 \right) = 0.5 \end{aligned}$$

Gini Index

- For **multi-class classification**, the Gini Index can be closer to **1**, depending on the number of classes and their distribution.
- The **Gini Index** is exactly **1** when the classes are **infinitely and equally distributed**
- **That means** that every possible class has an equal probability and there are infinitely many classes.
- However, in practical scenarios with a finite number of classes, the maximum Gini Index approaches but does not reach **1**.

Gini Index

- For example, let's assume an **infinitely large number of classes**, each with an equal probability:

$$p_1 = p_2 = p_3 = \dots = p_n = \frac{1}{n}$$

- Then, the Gini Index formula:

$$\begin{aligned} Gini &= 1 - \sum_{i=1}^n p_i^2 \\ &= 1 - \sum_{i=1}^n \left(\frac{1}{n}\right)^2 \\ &= 1 - n \times \frac{1}{n^2} \\ &= 1 - \frac{1}{n} \end{aligned}$$

As $n \rightarrow \infty$, we get: $Gini \rightarrow 1$

Gini Index

- **Real-World Example (Approaching 1 but Not Exactly)**
- For a **finite number of classes**, the highest possible Gini Index occurs when there are **many classes with equal probabilities**, but it never reaches exactly **1**.
- For example, if we have **4 classes** with equal probabilities:

$$p_1 = p_2 = p_3 = p_4 = 0.25$$

$$Gini = 1 - (0.25^2 + 0.25^2 + 0.25^2 + 0.25^2)$$

$$Gini = 1 - (4 \times 0.0625)$$

$$Gini = 1 - 0.25 = 0.75$$

- Thus, the Gini Index never actually reaches **1** in a practical scenario but gets closer as the number of equally probable classes increases.

Gini Index

- If a data set D contains examples from n classes, gini index, $gini(D)$ is defined as

$$gini(D) = 1 - \sum_{j=1}^n p_j^2$$

where p_j is the relative frequency of class j in D

- If a data set D is split on A into two subsets D_1 and D_2 , the $gini$ index $gini_A(D)$ is defined as

$$gini_A(D) = \frac{|D_1|}{|D|} gini(D_1) + \frac{|D_2|}{|D|} gini(D_2)$$

- Reduction in Impurity:

$$\Delta gini(A) = gini(D) - gini_A(D)$$

- The attribute provides the largest reduction in impurity is chosen to split the node.

Constructing CART (a Decision Tree) Using Gini Index

- Consider a dataset of loan applicants, where we want to predict if a person will default on a loan (**Yes** or **No**) based on two attributes: **Credit Score (High/Low)** and **Income (High/Low)**.

Credit Score	Income	Default (Target)
High	High	No
High	Low	No
Low	High	No
Low	Low	Yes
Low	Low	Yes
High	High	No

CART

Step 1: Compute Gini Index for the Root Node

The dataset has 4 "No" and 2 "Yes", so the Gini Index for the root node is:

$$\begin{aligned} Gini_{root} &= 1 - \left(\left(\frac{4}{6} \right)^2 + \left(\frac{2}{6} \right)^2 \right) \\ &= 1 - (0.444 + 0.111) = 1 - 0.555 = 0.445 \end{aligned}$$

Since the Gini Index is not zero, the dataset is impure, and we must split it.

CART

Step 2: Calculate Gini Index for Each Attribute Split

1. Splitting by "Credit Score"

- Credit Score = High $\rightarrow \{ \text{No, No, No} \} \rightarrow \text{Gini} = 0$
- Credit Score = Low $\rightarrow \{ \text{No, Yes, Yes} \}$

$$\begin{aligned} Gini_{Low} &= 1 - \left(\left(\frac{1}{3} \right)^2 + \left(\frac{2}{3} \right)^2 \right) \\ &= 1 - (0.111 + 0.444) = 1 - 0.555 = 0.445 \end{aligned}$$

Weighted Gini for Credit Score:

$$Gini_{CreditScore} = \left(\frac{3}{6} \times 0 \right) + \left(\frac{3}{6} \times 0.445 \right) = 0 + 0.222 = 0.222$$

CART

2. Splitting by "Income"

- Income = High $\rightarrow \{ \text{No, No, No} \} \rightarrow \text{Gini} = 0$
- Income = Low $\rightarrow \{ \text{No, Yes, Yes} \} \rightarrow \text{Gini} = 0.445$ (same as above)

Weighted Gini for Income:

$$Gini_{Income} = \left(\frac{3}{6} \times 0 \right) + \left(\frac{3}{6} \times 0.445 \right) = 0.222$$

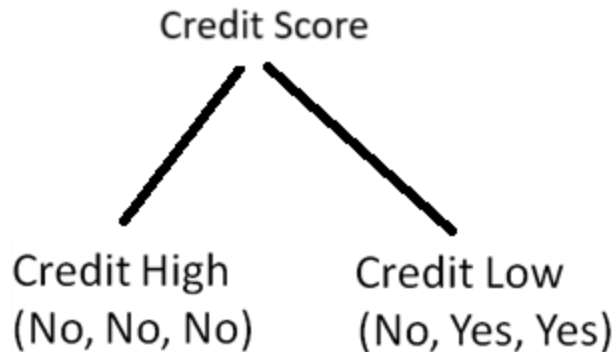
Step 3: Select the Best Attribute for Splitting

- $\text{Gini}(\text{Credit Score}) = 0.222$ and $\text{Gini}(\text{Income}) = 0.222$
- $\text{Reduction in Impurity}(\text{Credit Score}) = 0.445 - 0.222 = 0.223$
- $\text{Reduction in Impurity}(\text{Income}) = 0.445 - 0.222 = 0.223$

Since both attributes have the same Gini Index, so reduction in impurities are same $\Rightarrow$ we can choose any one, say **Credit Score** for the first splitting arbitrarily.

CART

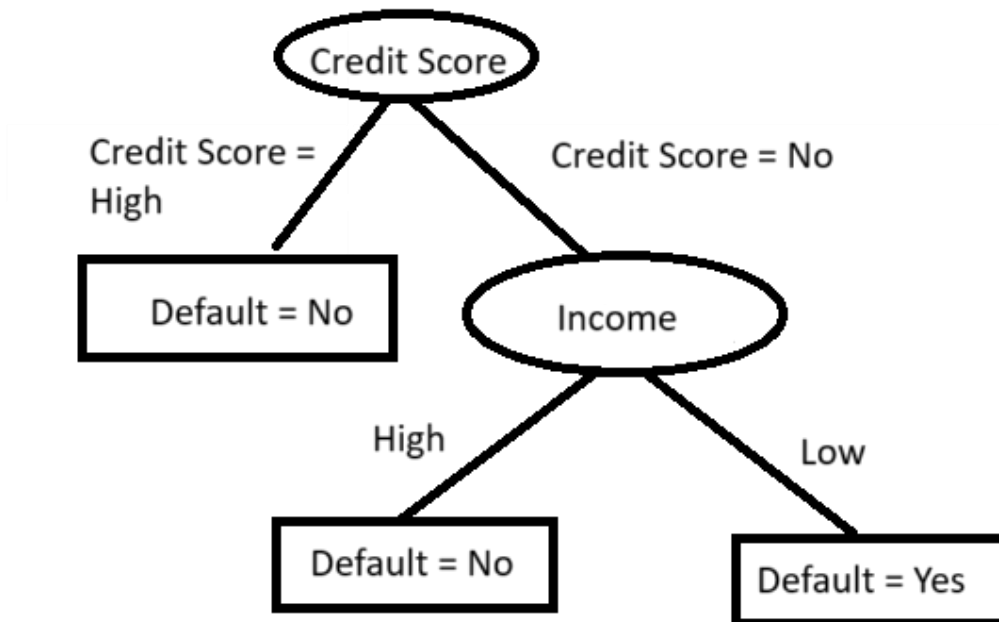
Step 4: Create the First Split in the Decision Tree



- **Left Node (High Credit Score)** is pure ($Gini = 0$), so it becomes a leaf node labeled **"No"**.
- **Right Node (Low Credit Score)** is still impure ($Gini = 0.445$), so we split it further.

CART

- **Step 5: Further Splitting on "Income" (For Low Credit Score Branch)**
- **Income = High** $\rightarrow \{ \text{No} \} \rightarrow \text{Gini} = 0$ (Pure)
- **Income = Low** $\rightarrow \{ \text{Yes, Yes} \} \rightarrow \text{Gini} = 0$ (Pure)
- Final Decision Tree



Which One to Use?

- **If computational efficiency is a priority → Use Gini Index (CART).**
- **If you want a more theoretically sound measure that accounts for information gain → Use Entropy (ID3, C4.5).**
- **Both perform similarly in practice, but Gini is preferred when speed is a concern.**

Information Gain VS Gini Index

1. Information Gain

- Based on the concept of **Entropy** from **Claude Shannon**.
- Measures **how much uncertainty is reduced** after splitting the dataset.
- Used mainly in algorithms like **ID3** and **C4.5**.
- The attribute with the **highest information gain** is selected for the split.
- **Idea:**
More reduction in entropy → Better split.

Information Gain VS Gini Index

2. Gini Index

- Measures **impurity of a dataset**.
- Used mainly in the **CART** algorithm.
- The attribute with the **lowest Gini index** is chosen for splitting.
- **Idea:**
Lower impurity → Better split.

Information Gain VS Gini Index

- **Information Gain:** measures reduction in randomness.
- **Gini Index:** measures how mixed the classes are.
- The **difference between uncertainty and impurity** is subtle but important. Both describe how **mixed or unpredictable the class labels are**, but they come from **different theoretical perspectives**.

Information Gain VS Gini Index

1. Uncertainty (Entropy / Information Gain)

- Comes from **Entropy** in **Information Theory**.
- Measures **how uncertain** we are about predicting the **class label**.
- If all samples belong to one class → **uncertainty = 0**.
- If classes are evenly mixed → **uncertainty is maximum**.
- **Example:**
- Dataset: 50% Yes, 50% No
- We are **very uncertain** about predicting the class.

Information Gain VS Gini Index

2. Impurity (Gini Index)

- Measures **how mixed the classes are within a node.**
- It represents the **probability of misclassifying a randomly chosen sample.**
- **Example:**
- Dataset: 50% Yes, 50% No
- High impurity because samples belong to different classes.

Information Gain VS Gini Index

- **Uncertainty:** “How unsure am I about the class label?”
- **Impurity:** “How mixed are the classes in this node?”
- In practice, both measure **how good a split is in a decision tree**, and they usually produce **very similar results**.
- In a **decision tree**, a **node** is a **point where the data is evaluated or split** based on a feature (attribute).

Information Gain VS Gini Index

- Each **node** contains a subset of the dataset.
- Measures like **Information Gain** and **Gini Index** are used to decide **how to split a node into child nodes**.
- A **node** is a point in the decision tree where **data is stored and a decision or split is made**.

Gini Index vs. Entropy

Feature	Entropy	Gini Index
Formula	$\text{Entropy} = -\sum p_i \log_2 p_i$	$\text{Gini} = 1 - \sum p_i^2$
Range	0 to $\log_2(n)$ (Binary case: 0 to 1)	0 to 1 (Binary case: 0 to 0.5)
Meaning	Measure of information gain (uncertainty)	Probability of incorrect classification
Impurity Interpretation	Lower Entropy means less randomness	Lower Gini means purer node
Computation	Slower (logarithm calculations)	Faster (no logarithm)
Decision Tree Algorithm	Used in ID3 , C4.5 , and C5.0	Used in CART (Classification and Regression Trees)
Splitting Preference	More sensitive to class distribution	Prefers larger class proportions (biased toward majority class)

Entropy is more sensitive to class distribution than Gini

- Entropy reacts more strongly to changes in class probabilities than Gini Index does.
- In other words, Entropy values **change more significantly** when class probabilities become more imbalanced, while Gini changes more gradually.

Entropy is more sensitive to class distribution than Gini

- **Example: Two-Class Classification (Binary Case)**
- Suppose we have a **node in a decision tree** with two classes: **Class A and Class B**. The probabilities of these classes are **p** and **1-p**.
- Then, $Entropy = -p \log_2(p) - (1 - p) \log_2(1 - p)$
and $Gini = 1 - (p^2 + (1 - p)^2)$
- Now, let's see how these values change for different probabilities:

p (Probability of class A)	Entropy	Gini Index
0.5	1.00	0.50
0.7	0.88	0.42
0.9	0.47	0.18
0.99	0.08	0.02

What does it Mean?

- **Entropy decreases more sharply as one class becomes dominant.**
 - At $p=0.9$, Entropy drops to **0.47**, while Gini is still **0.18**.
 - At $p=0.99$, Entropy is almost **0.08**, whereas Gini is **0.02**.
- **Gini Index decreases more gradually** as class imbalance increases.
- Thus, **Entropy is more sensitive** because it penalizes uncertainty more heavily.
- If the class distribution is highly imbalanced, Entropy drops significantly, while Gini decreases more smoothly.
- So, for highly imbalanced datasets, Gini Index is generally a better choice than Entropy.

For Highly Imbalanced Datasets

- **Entropy penalizes impurity more aggressively**, which can lead to deeper trees with more splits to separate the minority class completely.
- **Issue with Entropy in Imbalanced Datasets:**
 - Since Entropy reacts more strongly to class imbalance, it may create a **very deep tree** to minimize uncertainty.
 - This can lead to **overfitting**, where the tree models noise instead of patterns.

For Highly Imbalanced Datasets

- If your dataset is highly imbalanced:
 - **Use Gini Index** if you prioritize computational efficiency and want a balanced tree.
 - **Use Entropy** if you want to aggressively separate the minority class (but beware of overfitting).
- **Alternative Approaches to Handle Imbalance:**
Regardless of Gini or Entropy, you should also consider:
 - **Balancing the dataset** (e.g., SMOTE, oversampling, undersampling).
 - **Cost-sensitive learning** (assigning higher weights to the minority class).
 - **Ensemble** of decision trees
- **Pruning the decision tree** to prevent overfitting.

What is Overfitting in Decision Trees?

- Overfitting occurs when a decision tree model learns not only the underlying pattern in the training data but also **irrelevant details or noise**, making it **too complex and highly specific** to the training dataset.
- As a result, the model performs exceptionally well on training data but **generalizes poorly** to unseen test data, i.e., Poor accuracy for unseen samples.

Causes of Overfitting in Decision Trees

- **Excessive Depth (Too Many Splits)**
 - Decision trees tend to grow deep when they attempt to capture every minor variation in the training data, leading to an overly complex structure.
- **Small Leaf Nodes (Few Samples Per Leaf)**
 - If each leaf node contains very few training samples, the model memorizes the training data instead of learning a generalizable rule.
- **High Variance in the Data (Noisy Features)**
 - Decision trees are sensitive to noise, and they may **split excessively** based on noisy or irrelevant features rather than meaningful patterns.

Causes of Overfitting in Decision Trees

- **Imbalanced Datasets**
 - If certain classes dominate the dataset, the decision tree may **focus too much on those** and fail to generalize well for underrepresented classes.
- **Lack of Feature Selection**
 - Using **all available features** without evaluating their importance can lead to **redundant or irrelevant splits**, making the tree unnecessarily complex.

Methods to Prevent Overfitting in Decision Trees

1. Pruning (Post-Pruning and Pre-Pruning)

- Pruning is a technique used to **reduce the complexity** of a decision tree by **removing unnecessary branches** that do not contribute significantly to decision-making.
- **Pre-Pruning (Early Stopping)**
 - Stop the tree from growing too deep by limiting its complexity **during training**.
 - **Techniques:**
 - **Max Depth:** Set a limit on how deep the tree can grow.
 - **Min Samples per Leaf:** Specify the minimum number of samples required in a leaf node.
 - **Min Samples per Split:** Ensure a minimum number of samples required to split a node.

Methods to Prevent Overfitting in Decision Trees

- **Post-Pruning (Cost Complexity Pruning or Reduced Error Pruning)**
 - Grow the full tree first and then **remove branches** that do not improve test accuracy.
 - **Techniques:**
 - Use **cross-validation** to decide which branches to prune.
 - Set a pruning parameter α (alpha) in **Cost Complexity Pruning (CCP)** to penalize tree complexity.

What is Cost Complexity Pruning (CCP)?

- In decision trees, we don't just want accuracy—we also want **simplicity** (to avoid overfitting).
- CCP balances both using this formula:

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

- **R(T)** → Error of the tree (how wrong it is)
- **|T|** → Number of leaf nodes (tree size / complexity)
- **α** → Penalty for complexity

What is Cost Complexity Pruning (CCP)?

- What does α (alpha) actually do?
- Think of α as a “simplicity preference knob”:
- When $\alpha = 0$
 - No penalty for complexity
 - Tree grows as large as possible
 - Likely **overfitting**

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

What is Cost Complexity Pruning (CCP)?

- **When α is small (e.g., 0.001)**
 - Slight penalty
 - Keeps most splits
 - Mild pruning
- **When α is large**
 - Heavy penalty on tree size
 - Many branches removed
 - Simpler tree → may **underfit**

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

Underfitting

- Underfitting = model is too simple and fails to learn the data.
- **Underfitting** happens when a model is **too simple to capture the real patterns** in the data.
- It's like trying to fit a straight line to data that clearly curves.
- The model **misses important relationships**, so it performs poorly.
- **Key characteristics**
 - High error on **training data**
 - High error on **test data**
 - Model is **too simple**

Underfitting

- **In decision trees** Underfitting can happen if:
 - Tree is too shallow (low depth)
 - High min_samples_split
 - High min_samples_leaf
 - High pruning parameter (α)
- The tree becomes **too small** to capture patterns.
- **Fixing underfitting:** Increase model complexity
- For trees:
 - Increase depth
 - Reduce pruning (lower α)
 - Lower min_samples_leaf / min_samples_split

Parameters vs Hyperparameters in a Decision Tree

1. Parameters (learned from data)

- These are **automatically learned by the model during training**.
- In a decision tree, parameters include:
 - **Split rule** LIKE *feature A ≤ 5.3*
 - **Feature chosen at each node**
 - **Threshold values**
 - **Tree structure** (which nodes split, how deep it goes)
 - **Leaf predictions** (class labels or values)
- You **do NOT set these manually**—the algorithm figures them out.

Parameters vs Hyperparameters in a Decision Tree

2. Hyperparameters (set before training)

- These are **controls you set to guide how the tree is built.**
- **Common Decision Tree Hyperparameters**
 - **Tree size & shape**
 - **max_depth** → Maximum depth of the tree
 - **min_samples_split** → Minimum samples required to split a node
 - **min_samples_leaf** → Minimum samples required in a leaf
 - **max_leaf_nodes** → Maximum number of leaf nodes
 - **Pruning / Regularization: ccp_alpha (α)** → Controls pruning strength
(higher = simpler tree)

Methods to Prevent Overfitting in Decision Trees

2. Using Ensemble Methods

- Instead of relying on a single decision tree, ensemble methods build **multiple trees** and combine their results for better generalization.
- **Random Forest**
 - Constructs multiple decision trees and **averages their predictions**, reducing variance.
 - Uses **feature randomness** to prevent overfitting.
- **Gradient Boosting Trees (XGBoost, LightGBM, CatBoost)**
 - Boosting algorithms build multiple trees sequentially, each correcting errors of the previous tree.
 - Regularization methods help prevent overfitting.

Methods to Prevent Overfitting in Decision Trees

3. Feature Selection and Engineering

- Remove irrelevant or redundant features to simplify the tree.
- Use correlation analysis to identify and remove highly correlated features.

4. Regularization (Hyperparameter Tuning)

- Max Depth – Limits the depth of the tree.
- Min Samples per Split – Ensures a minimum number of samples are required to split a node.
- Min Samples per Leaf – Prevents the tree from creating too many small leaf nodes.
- Max Features – Limits the number of features considered for each split.

5. Cross-Validation for Hyperparameter Tuning

- Use k-fold cross-validation to evaluate model performance on different data splits and prevent overfitting.

Summary

Cause of Overfitting	Solution
Excessive Depth	Pruning (Pre-Pruning & Post-Pruning)
Small Leaf Nodes	Min Samples per Leaf
Noisy Features	Feature Selection & Regularization
Imbalanced Data	Data Balancing Techniques
Lack of Generalization	Ensemble Methods (Random Forest, Boosting)
No Validation Strategy	Cross-Validation

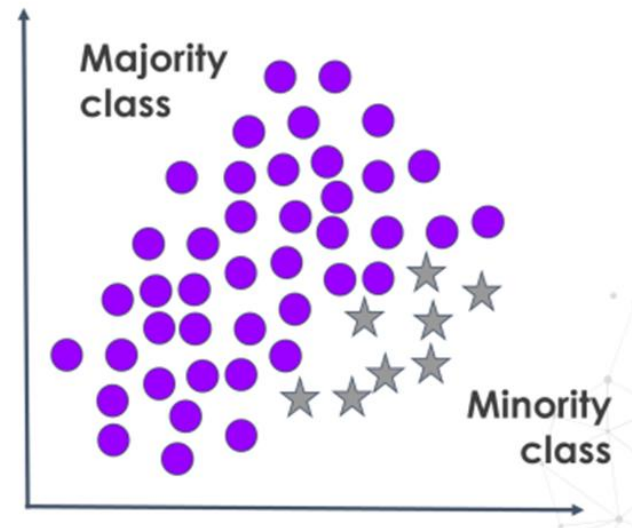
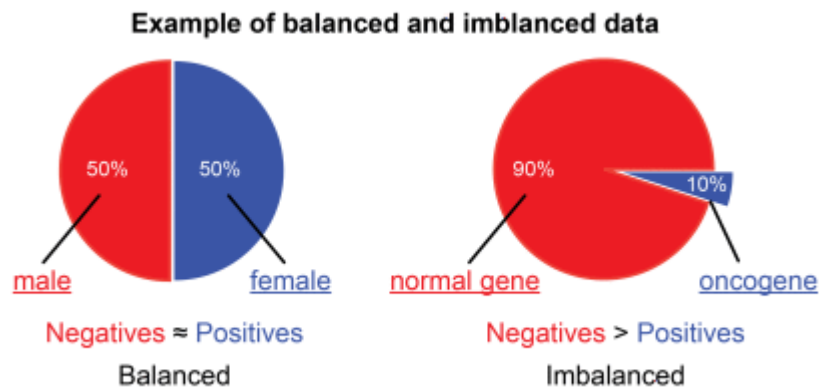
- By implementing **pruning, regularization, feature selection, and ensemble learning techniques**, we can effectively **reduce overfitting** in decision trees, ensuring better generalization to unseen data.

Other Attribute Selection Measures

- CHAID: a popular decision tree algorithm, measure based on χ^2 test for independence
- C-SEP: performs better than info. gain and gini index in certain cases
- G-statistic: has a close approximation to χ^2 distribution
- MDL (Minimal Description Length) principle (i.e., the simplest solution is preferred):
 - The best tree as the one that requires the fewest # of bits to both (1) encode the tree, and (2) encode the exceptions to the tree
- Multivariate splits (partition based on multiple variable combinations)
 - CART: finds multivariate splits based on a linear comb. of attrs.
- Which attribute selection measure is the best?
 - Most give good results, none is significantly superior than others

Introduction

- Class imbalance is a common problem in machine learning, especially in classification tasks, where the number of examples in one class is significantly higher or lower than in the other classes.
- A **class imbalanced dataset** is one in which the distribution of classes is not uniform. The dataset may contain one or more than one Minority class where the number of samples is significantly less than the Majority class.



Introduction

- Class imbalance is present in almost all real life data
 - Detection of Disease (Usually less number of data present in disease class)
 - Credit card fraud detection (Few number of fraudulent transactions)
 - Hardware fault detection (Usually number of fault is less)
 - Sentiment Detection (Data of particular sentiment can be less)
 - Traffic Accident prediction (Number of accident cases are less)

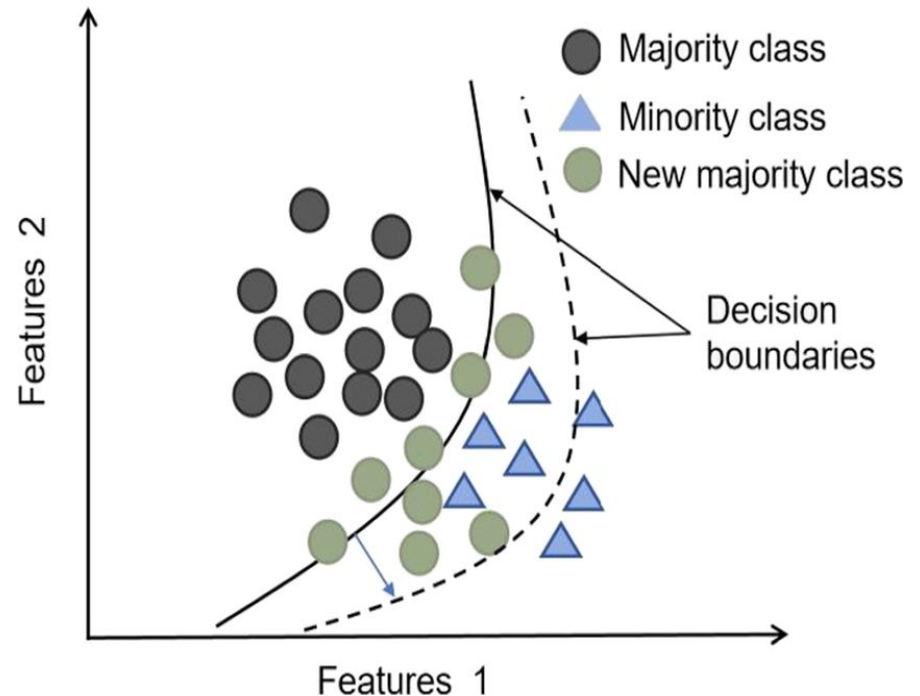
Challenges in Imbalanced Dataset

- (i) **Bias towards majority class:** Most models aim to achieve maximize the overall accuracy, so they tend to favor the majority class, resulting in poor performance on the minority class.
- (ii) **Underrepresentation of the Minority Class:** Critical features of the minority class may not be learned well due to insufficient samples.
- (iii) **Poor Metrics for Evaluation:** Accuracy alone is misleading. Precision, recall, F1-score, and other metrics are more informative in these scenarios.

Challenges in Imbalanced Dataset

(iv) Poor Decision

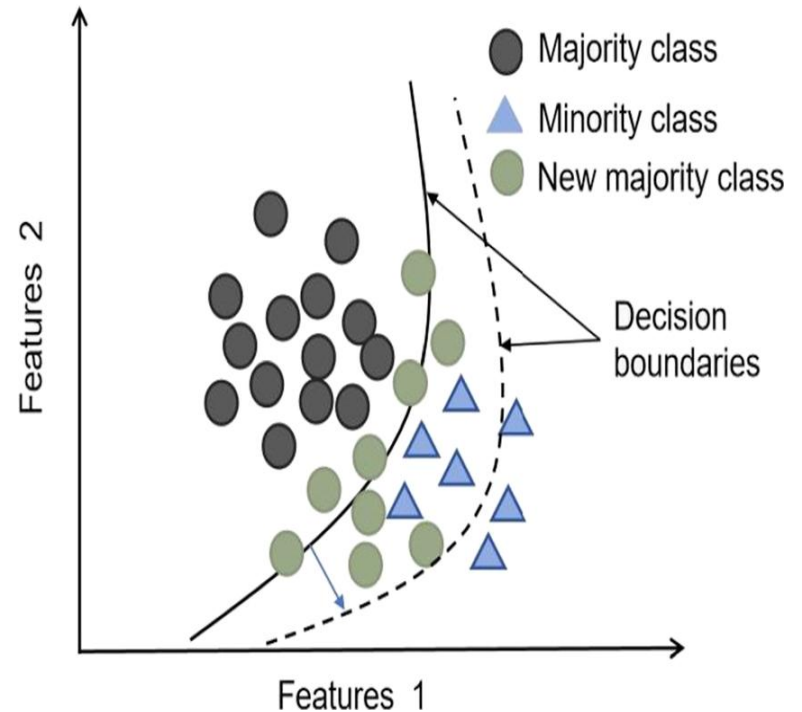
Boundaries: In imbalanced datasets, decision boundaries can be poorly defined, as the model learns to optimize for accuracy on the majority class, ignoring the correct classification of the minority class.



- When the new majority samples are added arbitrarily, they affect the decision boundaries

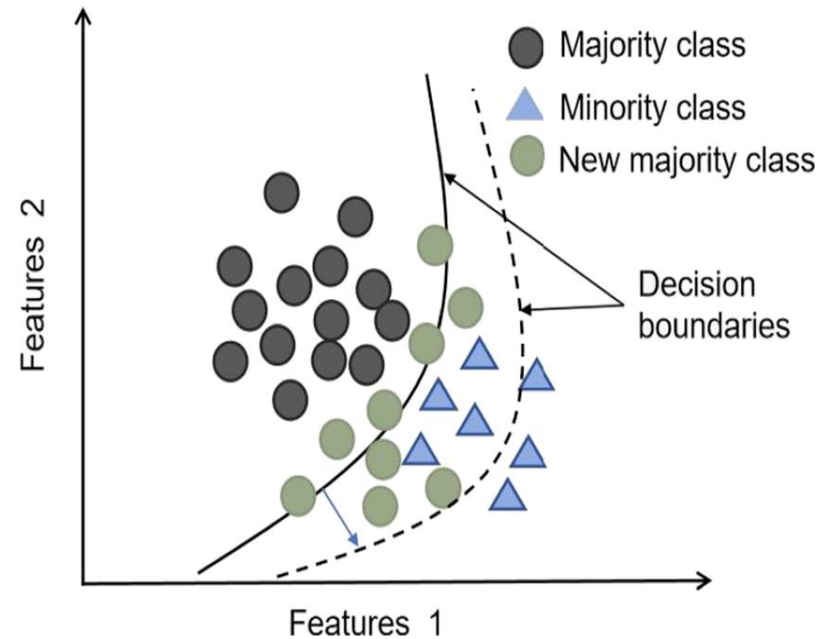
Challenges in Imbalanced Dataset

- The decision boundaries learned on majority samples tend to be increasingly biased towards the majority class, leading to the neglect of minority class data samples.
- This results:
 - in model overfitting for majority class samples
 - often leading to low generalization to minority class samples.



Challenges in Imbalanced Dataset

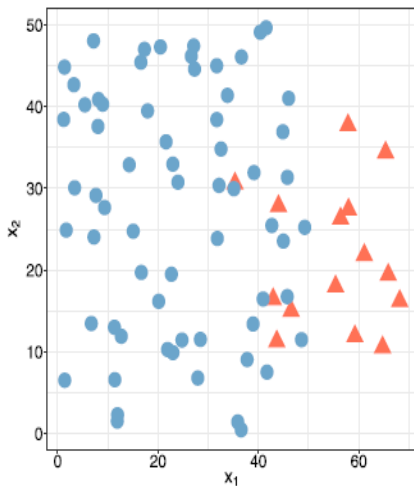
- Over time, the model may treat minority class patterns as noise, and misclassifying them because it has learned to prioritize the majority class patterns.



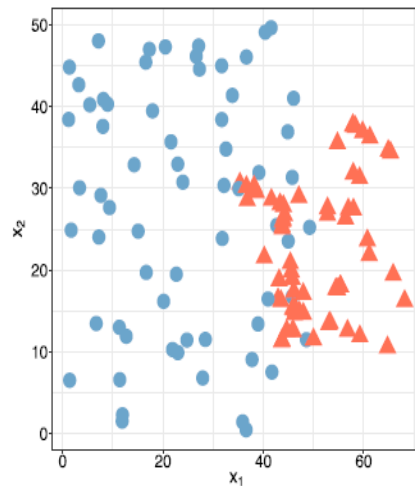
(vi) Over-reliance on Majority Samples in Loss Function: If the loss function does not take imbalance into account, it becomes heavily influenced by the majority class samples, reinforcing their patterns and resulting in overfitting.

Class Overlapping

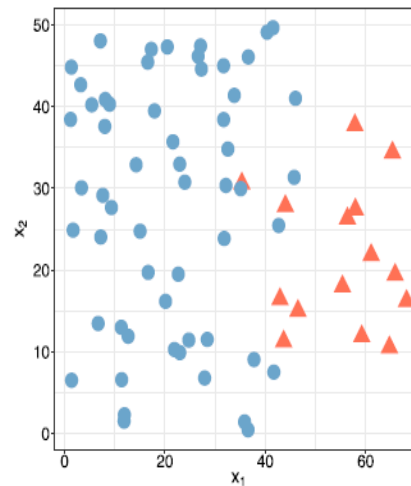
- Handling class imbalance is more difficult in presence of class overlapping.
- Overlapping has more effect on classifier performance than imbalance.



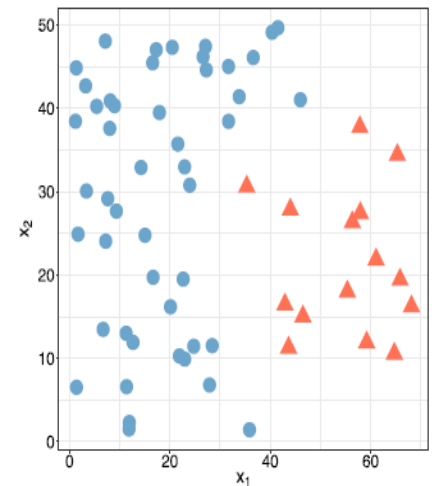
(a)



(b)

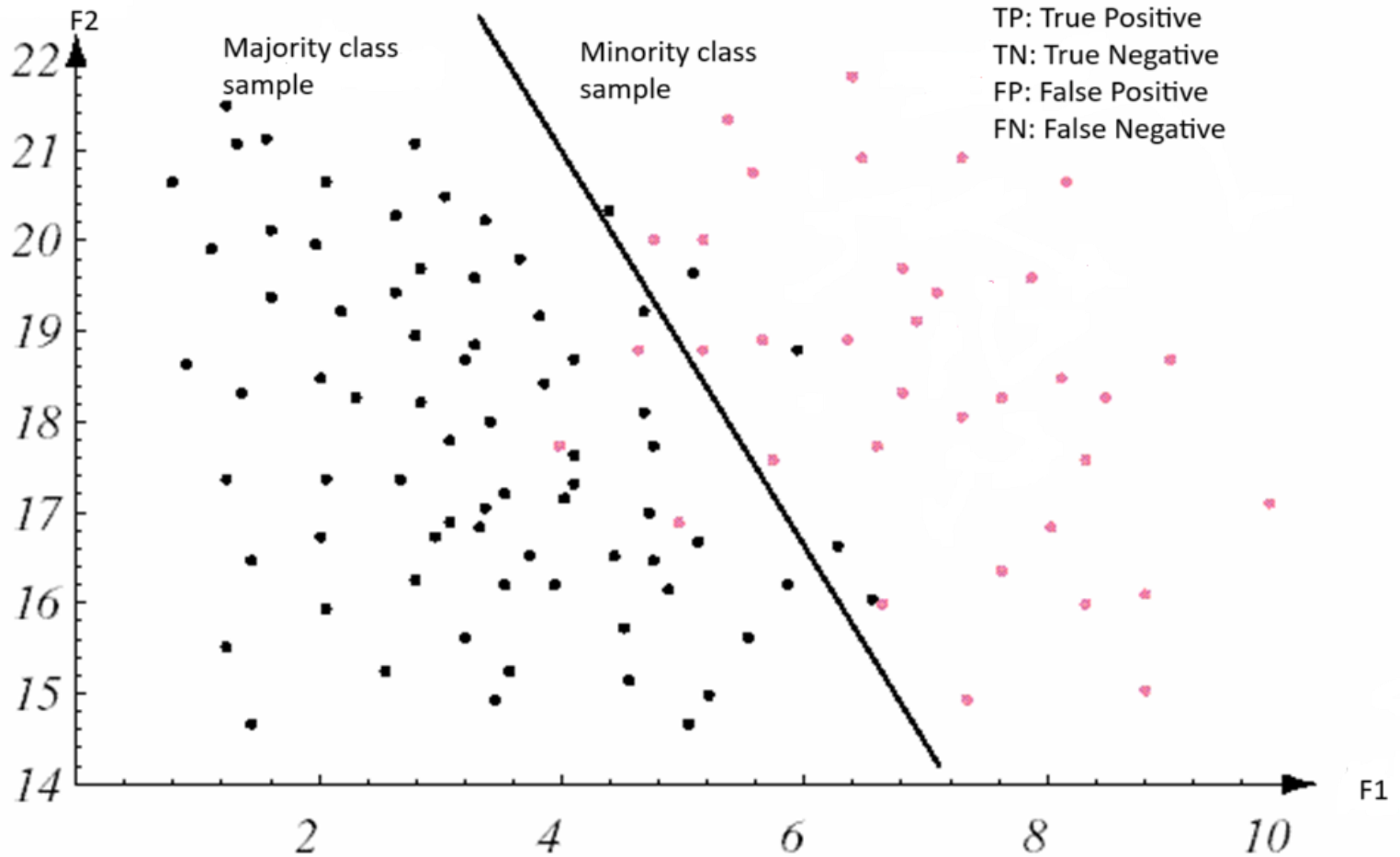


(c)



(d)

Classifier Evaluation



Classifier Evaluation Metrics: Confusion Matrix

Confusion Matrix	Predicted	
	Positive	Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **Precision:** exactness – what % of tuples that the classifier labeled as positive are actually positive?

$$Precision = \frac{TP}{TP + FP}$$

- **Recall:** completeness – what % of positive tuples did the classifier label as positive?

$$Recall = \frac{TP}{TP + FN}$$

- Perfect score of Precision and Recall is 1.0

Classifier Evaluation Metrics: Confusion Matrix

Confusion Matrix	Predicted	
	Positive	Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **Classifier Accuracy**, or recognition rate: percentage of test set tuples that are correctly classified

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

- **F-Measure (F_1 - Score)**: harmonic mean of precision and recall.

$$F - Measure = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Importance of Recall

Confusion Matrix	Predicted	
	Positive	Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- Let, a dataset contains 90 negative (majority) samples and 10 positive (minority) samples.
- Let a classifier gives TP=2, FN=8, FP=0, TN=90
- Here, Accuracy=92%, Precision = 100%, Recall=20%
- In a disease dataset, **Recall** is crucial because it's important to minimize false negatives, ensuring that most (if not all) actual cases are identified.
- Missing a disease case (false negative) could have severe consequences for patient health.

Importance of Precision

Confusion Matrix	Predicted	
	Positive	Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- Let, a dataset contains 10 positive (minority) samples and 90 negative (majority) samples.
- Let a classifier gives TP=10, FN=0, FP=50, TN=40
- Here, Accuracy=50%, Precision = $1/6=17\%$, Recall=100%
- **Precision** is very important in **fraud detection**.
- In this scenario, misclassifying an innocent transaction (negative sample) as fraudulent (false positive) can inconvenience customers and damage trust.
- High precision in fraud detection ensures that only truly fraudulent transactions are identified, minimizing unnecessary intervention.

Classifier Evaluation Metrics: Sensitivity and Specificity

Confusion Matrix	Predicted	
	Positive	Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- Sensitivity:** True Positive recognition rate = $\frac{TP}{P} = \frac{TP}{TP + FN}$
 = Recall

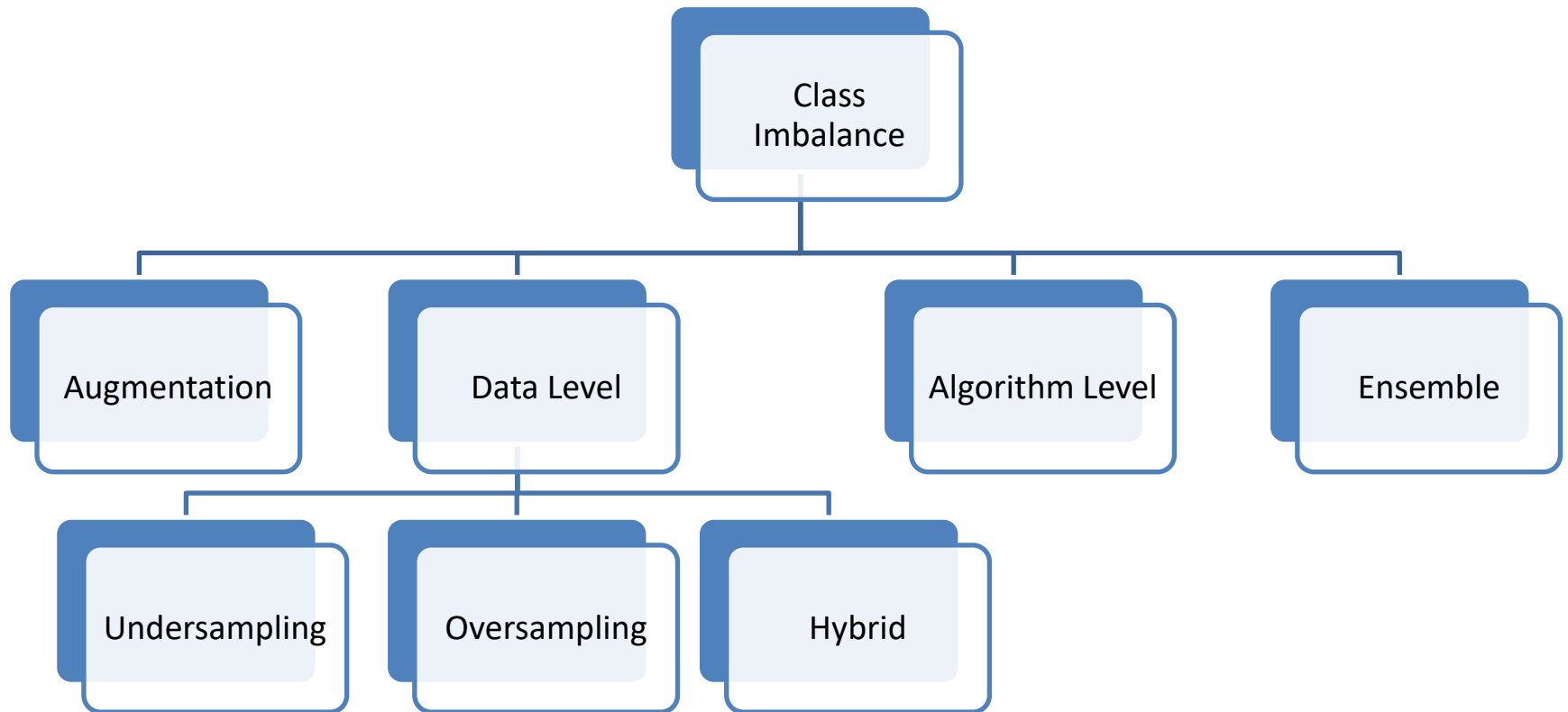
- Specificity:** True Negative recognition rate = $\frac{TN}{N} = \frac{TN}{TN + FP}$

Actual Class \ Predicted class	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	90	210	300	Sensitivity = 30.00
cancer = no	140	9560	9700	Specificity = 98.56
Total	230	9770	10000	Accuracy=96.40
Precision=90/230 = 39.13, Recall=90/300=30.00, F-Score=33.96				

Mitigation Strategies

- **Class Weights:** Adjust the loss function by assigning higher weights to minority class samples, making errors on minority samples more significant.
- **Resampling:** Use techniques like oversampling the minority class or undersampling the majority class to balance the dataset.
- **Use Advanced Models:** Ensemble methods, such as random forests or gradient boosting, can help by creating multiple classifiers that may handle the imbalance better.
- By applying these techniques, the model can achieve better generalization across classes and reduce the risk of overfitting to the majority class in imbalanced datasets.

Taxonomy of Existing Solutions for Class Imbalance



Data Level Methods for Class Imbalance

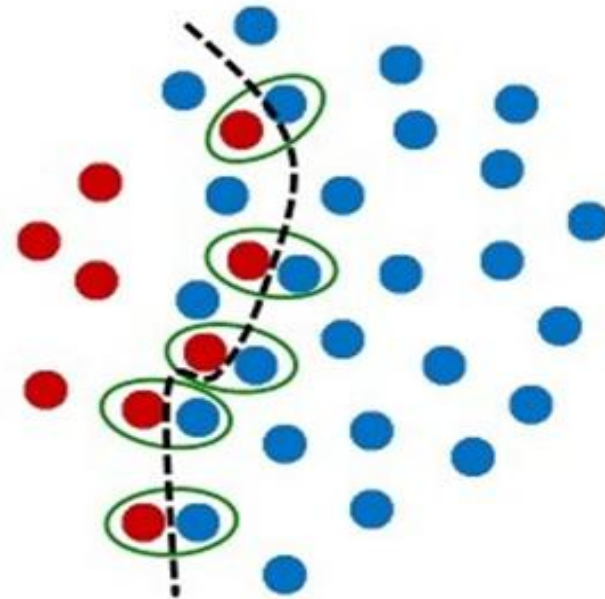
Undersampling	Oversampling	Hybrid
• Random Undersampling • Condensed Nearest Neighbor • Tomek Links • One Sided Selection • Neighborhood Cleaning Rule • Undersampling Based on Clustering	• Random Oversampling • SMOTE • Borderline-SMOTE • Safe Level SMOTE • ADASYN • CLUSBUS	• SMOTE-ENN • SMOTE-Tomek • SPIDER

Undersampling: Tomek links

- Tomek Links are a technique used in imbalanced classification problems to remove borderline or noisy instances, helping improve classifier performance.
- They were introduced by Ivan Tomek in 1976 as a method for data cleaning.
- A **Tomek Link** exists between two instances (x_1 , x_2) from different classes if: x_1 is the nearest neighbor of x_2 , and x_2 is the nearest neighbor of x_1 .

Undersampling: Tomek link

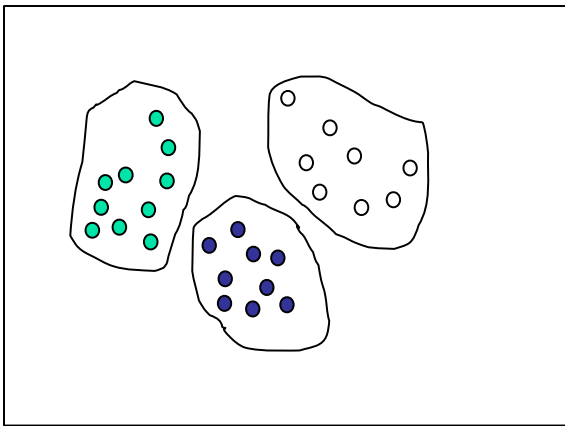
- Tomek links are used to:
 - (i) **Identify and remove overlapping instances:** By eliminating majority class instance in each Tomek link, one can reduce the overlap between classes, leading to better-defined class boundaries.



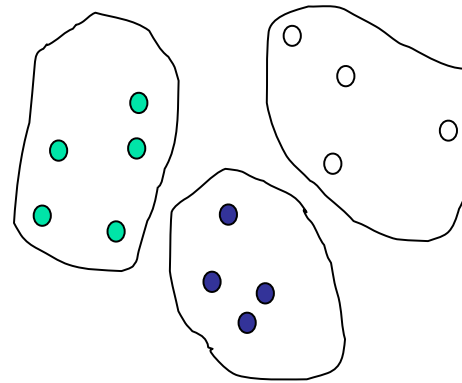
- (ii) **Clean the dataset:** The idea is that such pairs lie close to the decision boundary and might represent **noise or borderline cases**. Removing Tomek links helps in getting rid of noisy instances that may adversely affect the classifier's performance.

Undersampling: Based on Clustering

- Sampling: obtaining a small sample s to represent the whole data set N
- Cluster the majority samples into different groups
- Stratified sampling: Approximate the percentage of each group in the overall dataset so that the distribution in each group is approximately the same as that in the initial set



Cluster the majority samples



Apply the stratify sampling

Oversampling: SMOTE

- **SMOTE (Synthetic Minority Over-sampling Technique)** is a popular method used in machine learning to address the problem of imbalanced datasets.
- **Selecting Minority Class Samples:** For each minority sample, it finds its k-nearest neighbors in the minority class.
- It picks one of the neighbors randomly and creates a synthetic sample along the line segment between the minority sample and its chosen neighbor.
- The synthetic sample is generated as a weighted combination of the two points:

$$\text{new}_{\text{sample}} = \text{sample} + \text{random}(0, 1) \times (\text{neighbor} - \text{sample})$$

Key Points in SMOTE

- **Synthetic Sampling:** Unlike simple oversampling, SMOTE doesn't just duplicate samples. Instead, it creates new, synthetic samples that are similar but not identical to existing ones.
- **Better Generalization:** SMOTE often leads to better model generalization because the model gets a more balanced view of the minority class.
- **Use with Caution:** SMOTE may sometimes introduce noise, especially if the minority class samples are spread across a wide feature space or overlap significantly with the majority class.

Oversampling : **Borderline-SMOTE**

- **Borderline-SMOTE**: It is a variation of SMOTE designed to **focus on samples near the decision boundary** rather than generating synthetic data randomly across the minority class.
- This improves model performance by **strengthening the decision boundary**.

Borderline-SMOTE

1. Identify Borderline Instances

- For each minority-class instance, find its **k-nearest neighbors**.
- Count how many of these neighbors belong to the **majority class**.
- If **most of the k-nearest neighbors** are from the majority class, the minority instance is considered borderline instances which are "**hard to classify**" (i.e., close to the decision boundary).
- These borderline instances are selected for oversampling.

Borderline-SMOTE

2. Generate Synthetic Samples

- Similar to SMOTE, synthetic points are created **only for borderline instances**.
- A new sample is generated along the line segment between the borderline instance and one of its k minority-class neighbors:

$X_{\text{new}} = X_{\text{original}} + \lambda \times (X_{\text{neighbor}} - X_{\text{original}})$, where λ is a random number between 0 and 1.

Borderline-SMOTE

- There are two Types of Borderline-SMOTE
 - (i) Borderline-SMOTE1:** Only oversamples the identified borderline minority instances.
 - (ii) Borderline-SMOTE2:** Generates more synthetic points **between the minority instances and majority-class neighbors**, further strengthening the decision boundary.

Differences between Borderline-SMOTE1 vs. Borderline-SMOTE2

Feature	Borderline-SMOTE1	Borderline-SMOTE2
Where synthetic samples are generated	Only between borderline minority instances and their minority neighbors	Between borderline minority instances and both minority & majority neighbors
Aggressiveness	Moderate (focuses on reinforcing decision boundary)	More aggressive (pushes decision boundary into majority class)
Risk	Lower (less chance of generating misleading samples)	Higher (may create synthetic samples too close to majority class, increasing overlap)

Hybrid Sampling

1. SMOTE-Tomek

- Tomek links are often used in combination with other techniques to address class imbalance.
- For instance, the **SMOTE-Tomek Links** method first applies the Synthetic Minority Over-sampling Technique (SMOTE) to generate synthetic samples for the minority class and
- Then uses Tomek links to clean the resulting dataset by removing overlapping instances.

Hybrid Sampling

2. SMOTE-ENN:

- SMOTE-ENN (Synthetic Minority Over-sampling Technique + Edited Nearest Neighbors) is a hybrid sampling technique.
- It first oversamples the minority class using SMOTE and then cleans the dataset using Edited Nearest Neighbors (ENN).
- This combination helps improve class balance while removing noisy and borderline instances, leading to better classification performance.

How SMOTE-ENN Works

Step 1: Apply SMOTE (Oversampling)

- SMOTE creates synthetic minority class samples by interpolating between existing minority instances.
- This helps balance the dataset by increasing the number of minority samples.

Step 2: Apply ENN (Undersampling)

- ENN removes samples (both majority and minority class instances) that are misclassified by their k-nearest neighbors (k-NN, usually $k=3$).

Advantages of SMOTE-ENN

- **Better class balance** – Increases minority samples while also removing irrelevant majority samples.
- **Reduces noise** – ENN removes misclassified or ambiguous points, improving data quality.
- **Stronger decision boundary** – Helps classifiers perform better by removing confusing samples.

Deep Learning based Oversampling

- Deep learning techniques can be useful for oversampling, particularly when dealing with complex, high-dimensional data where traditional oversampling methods (like SMOTE) might fall short.
- Deep learning-based oversampling approaches are more sophisticated and can generate realistic synthetic samples, preserving the complexity of the minority class distribution.
- The following deep learning techniques are applied in oversampling:
 - (i) Generative Adversarial Networks (GANs)
 - (ii) Variational Autoencoders (VAEs)
 - (iii) Reinforcement Learning for Data Augmentation
 - (iv) Autoencoders with Clustering Techniques

Thank You